

COMMUNITY LIBRARY MANAGEMENT SYSTEM

Requirement Understanding Document

Project Type

.NET 8 Console Application with Entity Framework Core & PostgreSQL

Architecture

Three-Tier: DataAccessLayer | BusinessLayer | PresentationLayer

Approach

Database First with EF Core Scaffolding

Prepared By

Iniyan

Date

18 May 2026

1. Project Overview

1.1 Project Summary

The Community Library Management System is a console-based application built using .NET 8 and Entity Framework Core with PostgreSQL as the backend database. The system is designed to digitize and automate the daily operations of a small community library, including member registration, book cataloging, borrowing, returns, and fine management.

1.2 Objectives

- Provide a fully functional library management system through a console interface
- Implement member management with different membership tiers
- Enable efficient book catalog and copy management
- Enforce business rules for borrowing and returns
- Automate fine calculation for late returns
- Generate useful reports for library administration
- Demonstrate EF Core capabilities including transactions and PostgreSQL functions

1.3 Technology Stack

Component	Technology	Version	Purpose
Runtime	.NET	8.0	Application platform
Language	C#	12.0	Primary programming language
ORM	Entity Framework Core	8.0.0	Database access and mapping
Database	PostgreSQL	Latest	Primary data store
DB Provider	Npgsql EF Core Provider	8.0.0	EF Core PostgreSQL adapter
DI Container	Microsoft.Extensions.DI	8.0.0	Dependency injection
Testing	xUnit	Latest	Unit testing framework
Mocking	Moq	4.20.70	Test mocking library
Assertions	FluentAssertions	6.12.0	Fluent test assertions
In-Memory DB	EF Core InMemory	8.0.0	Testing database

1.4 Architecture Overview

The application follows a Three-Tier Architecture pattern separating concerns into distinct layers:

Layer	Project Name	Responsibility
Presentation Layer	PresentationLayer	Console UI, menus, user input/output, logging

Layer	Project Name	Responsibility
Business Layer	BusinessLayer	Business rules, validation, DTOs, service logic
Data Access Layer	DataAccessLayer	EF Core models, DbContext, repositories, DB operations

2. Functional Requirements

2.1 Member Management

The system manages library members with the following capabilities:

2.1.1 Features

#	Feature	Description	Status
1	Add New Member	Register a new member with name, phone, email and membership type	✓ Done
2	View All Members	Display all registered members in a tabular format	✓ Done
3	Search by Email	Find a specific member using their email address	✓ Done
4	Search by Phone	Find a specific member using their phone number	✓ Done
5	Update Status	Change member status between Active and Inactive	✓ Done
6	Deactivate Member	Deactivate a member preventing further borrowing	✓ Done
7	View Lendings	View all active borrows of a specific member with details	✓ Done

2.1.2 Membership Types

Type	Max Active Borrowings	Max Borrow Days	DB Value
Basic	2 Books	7 Days	0
Student	3 Books	10 Days	1
Premium	5 Books	15 Days	2

2.1.3 Member Validation Rules

- Name must be at least 2 characters, letters and spaces only
- Phone must be exactly 10 digits (validated with regex: `^\d{10}$`)
- Email must follow standard format (validated with regex)
- Duplicate email and phone are rejected with friendly error messages

2.2 Book Management

The system manages the library's book catalog and physical copies:

2.2.1 Features

#	Feature	Description	Status
1	Add New Book	Add a book with ISBN, title, author, and category	✓ Done

#	Feature	Description	Status
2	Add Book Copy	Add one or more physical copies of an existing book	✓ Done
3	View All Books	Display all books with total and available copy counts	✓ Done
4	Search Books	Search by title, author, or category keyword	✓ Done
5	View by Category	Filter and display books belonging to a specific category	✓ Done
6	Update Copy Status	Mark a copy as Available, Damaged, or Lost	✓ Done
7	Add Category	Create new book categories	✓ Done
8	View Categories	List all available book categories	✓ Done

2.2.2 Book Copy Status Values

Status	DB Value	Description
Available	0	Copy is on the shelf and can be borrowed
Borrowed	1	Copy is currently with a member
Damaged	2	Copy is damaged and not available for borrowing
Lost	3	Copy is lost and removed from circulation

2.3 Borrowing System

The borrowing module enforces strict business rules before allowing a member to borrow a book:

2.3.1 Borrowing Validation Rules

#	Rule	Condition	Error Thrown
1	Member Exists	Member ID must exist in the database	MemberNotFoundException
2	Member Active	Member status must be Active	InactiveMemberException
3	Fine Check	Unpaid fine must not exceed ₹500	UnpaidFineException
4	Borrow Limit	Active borrows must be less than membership limit	BorrowLimitExceededException
5	Duplicate Check	Member must not have same book already borrowed	DuplicateBorrowException
6	Copy Availability	At least one copy must have Available status	BookNotAvailableException

2.3.2 Borrowing Transaction Flow

All borrowing operations are wrapped in an EF Core transaction to ensure data integrity:

1. Validate Member ID format (InputValidator)
2. Validate Book ID format (InputValidator)
3. Begin EF Core database transaction
4. Fetch member and verify existence → throw MemberNotFoundException if not found
5. Check member status → throw InactiveMemberException if inactive
6. Calculate unpaid fine → throw UnpaidFineException if fine > ₹500
7. Count active borrows → throw BorrowLimitExceededException if at limit
8. Check duplicate active borrow for same book → throw DuplicateBorrowException
9. Find available book copy → throw BookNotAvailableException if none
10. Create Borrow record with calculated due date
11. Update BookCopy status to Borrowed
12. Commit transaction → on any failure, rollback entire operation

2.4 Return System

2.4.1 Return Flow

13. Member enters their Member ID
14. System displays all active borrows with Borrow ID, Book Title, Due Date, and Status
15. Member selects the Borrow ID to return
16. System validates the borrow belongs to the member
17. If overdue: calculate fine ($\text{₹}10 \times \text{overdue days}$)
18. If fine exists: member must pay exact fine amount before return is processed
19. After fine payment: confirm return with member
20. System marks borrow as Returned, saves return date and fine amount

21. System marks book copy as Available again

2.4.2 Fine Calculation

Fine Formula: Fine Amount = Number of Overdue Days × ₹10 per day Example: 5 days late × ₹10 = ₹50 fine Payment: Exact amount must be paid — no partial payments for return

2.5 Fine Management

#	Feature	Description	Status
1	View Pending Fine	Show total unpaid fine for a member using PostgreSQL function	✓ Done
2	Pay Fine	Record fine payment, validate exact or partial amounts	✓ Done
3	View Fine History	Show all past fine payments for a member	✓ Done
4	Borrow Block	Prevent borrowing when unpaid fine exceeds ₹500	✓ Done
5	Return Fine Pay	Collect exact overdue fine before processing book return	✓ Done

2.6 Reports

#	Report Name	Description	Status
1	Currently Borrowed Books	All active borrows with member, book, due date and overdue status	✓ Done
2	Overdue Books	All borrows past due date with days late and estimated fine	✓ Done
3	Member Borrowing Summary	Active borrows, return history and unpaid fines for a member	✓ Done
4	Available Books by Category	Books with available copies filtered by category	✓ Done
5	Members with Pending Fines	All members who have unpaid fines, sorted by amount descending	✓ Done
6	Most Borrowed Books	Top 10 books ranked by total borrow count	✓ Done
7	Member Borrowing History	Complete borrow and return history for a specific member	✓ Done

3. Non-Functional Requirements

3.1 Validation

Field	Validation Rule	Implementation
Name	Min 2 chars, letters and spaces only	Regex: <code>^[a-zA-Z\s]+\$</code>
Phone	Exactly 10 digits	Regex: <code>^\d{10}\$</code>
Email	Standard email format	Regex: <code>^[^@\s]+@[^\s]+\.[^\s]+\$</code>
ISBN	Max 20 characters, not empty	String length check
Title	Max 255 characters, not empty	String length check
Author	Max 150 characters, not empty	String length check
Category	Max 100 characters, not empty	String length check
ID Fields	Must be positive integer	id > 0 check
Amount	Must be greater than zero	amount > 0 check

3.2 Exception Handling

The application uses a custom exception hierarchy for meaningful error messages:

Exception Class	When Thrown	Layer
LibraryException	Base class for all library exceptions	Business
InvalidInputException	User input fails validation rules	Business
MemberNotFoundException	Member ID does not exist in database	Business
BookNotFoundException	Book ID does not exist in database	Business
BookNotAvailableException	No available copies for the requested book	Business
InactiveMemberException	Member status is Inactive	Business
BorrowLimitExceededException	Member has reached their borrowing limit	Business
UnpaidFineException	Member's unpaid fine exceeds ₹500 limit	Business
DuplicateBorrowException	Member already has active borrow for same book	Business
AlreadyReturnedException	Attempting to return an already returned borrow record	Business

3.3 Transaction Management

EF Core transactions are used for the two critical operations:

- Borrow Book: 6-step transaction covering validation, record creation, and copy status update
- Return Book: 3-step transaction covering fine calculation, borrow update, and copy release

- Any exception during transaction triggers automatic rollback

3.4 Session Logging

Every session is automatically logged to a timestamped file:

- Log file created at application startup in the Logs/ directory
- All user inputs are logged with field name and value
- All successes, errors, and section headers are logged
- Session start and end timestamps are recorded
- Log file path displayed to user on exit

4. Database Design

4.1 Design Approach

Approach: Database First — SQL schema was designed manually and tables created in PostgreSQL. EF Core Scaffold command was used to generate model classes and DbContext automatically from the existing database.

4.2 Enum Strategy

All enum-type columns are stored as INT in the database and mapped to C# enums in the application layer. This avoids PostgreSQL-specific enum types and keeps the schema portable:

C# Enum	Column	Table	Values
MembershipTypeEnum	type	membership_type	0=Basic, 1=Student, 2=Premium
MemberStatus	status	members	0=Active, 1=Inactive
CopyStatus	status	book_copies	0=Available, 1=Borrowed, 2=Damaged, 3=Lost
BorrowStatus	status	borrow	0=Borrowed, 1=Returned

4.3 Tables

4.3.1 category

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment identifier
name	VARCHAR(100)	NOT NULL, UNIQUE	Category name (Fiction, Science, etc.)

4.3.2 membership_type

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment identifier
type	INT	NOT NULL, UNIQUE	Membership type enum value
max_borrowings	INT	NOT NULL	Maximum concurrent borrows allowed
max_borrow_days	INT	NOT NULL	Maximum days a book can be borrowed

4.3.3 members

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment identifier
name	VARCHAR(150)	NOT NULL	Member full name
phone	VARCHAR(15)	NOT NULL, UNIQUE	10-digit phone number
email	VARCHAR(150)	NOT NULL, UNIQUE	Email address
membership_type_id	INT	FK → membership_type	Member's membership category
status	INT	NOT NULL, DEFAULT 0	0=Active, 1=Inactive
joined_date	DATE	NOT NULL	Date member registered

4.3.4 book

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment identifier
isbn	VARCHAR(20)	NOT NULL UNIQUE	International Standard Book Number
title	VARCHAR(255)	NOT NULL	Book title
author	VARCHAR(150)	NOT NULL	Author name
category_id	INT	FK → category	Book's category

4.3.5 book_copies

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment identifier
book_id	INT	FK → book	Which book this copy belongs to
status	INT	DEFAULT 0	0=Available, 1=Borrowed, 2=Damaged, 3=Lost
remarks	VARCHAR(300)	NULLABLE	Optional notes about this copy's condition

4.3.6 borrow

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment identifier
member_id	INT	FK → members	Who borrowed the book
book_copy_id	INT	FK → book_copies	Which physical copy was borrowed
date_of_borrow	DATE	NOT NULL	Date the book was borrowed
due_date	DATE	NOT NULL	Date the book must be returned by
date_of_return	DATE	NULLABLE	Actual return date (null if not returned)
fine_amount	NUMERIC(10,2)	DEFAULT 0	Calculated fine for late return

Column	Type	Constraints	Description
status	INT	DEFAULT 0	0=Borrowed, 1=Returned

4.3.7 fine_payments

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment identifier
borrow_id	INT	FK → borrow	Which borrow record this payment is for
member_id	INT	FK → members	Who made the payment
amount_paid	NUMERIC(10,2)	NOT NULL	Amount paid in this transaction
payment_date	TIMESTAMP WITHOUT TIME ZONE	NOT NULL	When the payment was made

4.4 PostgreSQL Function

As required, one PostgreSQL function was created and called from EF Core:

Function: calculate_member_fine(p_member_id INT) Returns: NUMERIC — Total unpaid fine for a member Logic: SUM(borrow.fine_amount) - SUM(fine_payments.amount_paid) Called from: FineRepository.GetTotalUnpaidFineAsync() with manual calculation fallback

5. Project Structure

5.1 Solution Layout

Project	Type	Purpose
DataAccessLayer	Class Library	Models, DbContext, Repositories, Enums
BusinessLayer	Class Library	Services, Interfaces, DTOs, Validators, Exceptions
PresentationLayer	Console App	Menus, Program.cs, ConsoleHelper, Logger
Tests	xUnit Test Project	Unit tests for all service and validator classes

5.2 DataAccessLayer Structure

Folder	Contents
Context/	LibraryDbContext.cs — EF Core DbContext (scaffolded)
Models/	Book, BookCopy, Borrow, Category, FinePayment, Member, MembershipType
ModelExtensions/	Partial classes adding enum properties to scaffolded models
Enums/	BorrowStatus, CopyStatus, MembershipTypeEnum, MemberStatus
Repositories/	IMemberRepository, IBookRepository, IBorrowRepository, IFineRepository, ICategoryRepository
Repositories/	MemberRepository, BookRepository, BorrowRepository, FineRepository, CategoryRepository

5.3 BusinessLayer Structure

Folder	Contents
Interfaces/	IMemberService, IBookService, IBorrowService, IFineService
Services/	MemberService, BookService, BorrowService, FineService
DTOs/	MemberDto, BookDto, BookCopyDto, BorrowDto, BorrowingSummaryDto, FineDto, FinePaymentDto, CategoryDto
Validators/	InputValidator — static class with regex-based field validations
Exceptions/	LibraryException, InvalidInputException, MemberNotFoundException, BookNotFoundException, BookNotAvailableException, InactiveMemberException, BorrowLimitExceededException, UnpaidFineException, DuplicateBorrowException, AlreadyReturnedException

5.4 PresentationLayer Structure

File/Folder	Contents
Program.cs	DI container setup, service registration, main menu loop
ConsoleHelper.cs	Static helper for colored output, input reading, exception display, and logging
Logging/	FileLogger.cs — timestamped session file logger
Menus/	MemberMenu, BookMenu, BorrowMenu, FineMenu, ReportMenu

5.5 Tests Structure

File	Tests Count	What is Tested
InputValidatorTests.cs	30+	Phone regex, email regex, name rules, ID validation, amount validation
MemberServiceTests.cs	15+	Add member, duplicate checks, search, deactivate, status update
BookServiceTests.cs	20+	Add book, add copy, search, category filter, copy status update
BorrowServiceTests.cs	9+	Borrow validations, return flow, fine calculation, transaction
FineServiceTests.cs	12+	Fine summary, pay fine, fine history, pay for borrow

6. Business Rules Summary

6.1 Borrowing Rules

Rule	Details
Membership Limit	Basic: 2 books max Student: 3 books max Premium: 5 books max
Duration Limit	Basic: 7 days Student: 10 days Premium: 15 days
Inactive Block	Member with Inactive status cannot borrow any books
Fine Block	Member with unpaid fine over ₹500 cannot borrow until fine is paid
Duplicate Block	Member cannot borrow the same book twice if the first copy is not returned
Availability Check	A book can only be borrowed if at least one copy has Available status

6.2 Return Rules

Rule	Details
Fine Calculation	₹10 per day for each day beyond the due date
Fine Payment	For overdue returns, exact fine must be paid before book is returned
Copy Release	On successful return, book copy status changes back to Available
Record Update	Borrow status set to Returned, return date and fine amount saved
No Partial Pay	Partial fine payment for return is not allowed — exact amount required

6.3 Fine Rules

Rule	Details
Fine Rate	₹10 per overdue day
Borrow Block	Unpaid fine over ₹500 blocks new borrowing
Payment Tracking	All fine payments recorded in fine_payments table with timestamp
History	Members can view their complete payment history
Return Flow	Fines from return are paid immediately before return is confirmed

7. Requirement Coverage

7.1 Functional Requirements Coverage

#	Requirement	Implementation	Status
1	Add new member	MemberService.AddMemberAsync() with full validation	✓ Done
2	View all members	MemberService.GetAllMembersAsync() → MemberMenu display	✓ Done
3	Search by phone/email	MemberService.GetByPhone/Email with regex validation	✓ Done
4	Update membership status	MemberService.UpdateMemberStatusAsync()	✓ Done
5	Deactivate a member	MemberService.DeactivateMemberAsync()	✓ Done
6	Membership types	MembershipType table with seeded Basic/Student/Premium rows	✓ Done
7	Add new book	BookService.AddBookAsync() with ISBN and category check	✓ Done
8	Add multiple copies	BookService.AddBookCopyAsync()	✓ Done
9	View available books	BookService.GetAllBooksAsync() with available copy count	✓ Done
10	Search books	BookRepository.SearchAsync() by title/author/category	✓ Done
11	Mark copy damaged/unavailable	BookService.UpdateCopyStatusAsync() with CopyStatus enum	✓ Done
12	Enforce borrowing limits	BorrowService with BorrowLimitExceededException	✓ Done
13	Inactive member block	BorrowService checks member.Status before borrowing	✓ Done
14	Fine > ₹500 block	BorrowService checks unpaid fine before borrowing	✓ Done
15	Duplicate borrow block	BorrowService.GetActiveBorrowByMemberAndBook check	✓ Done
16	Late return fine (₹10/day)	BorrowService.ReturnBookAsync() calculates overdue days	✓ Done
17	Copy available after return	BorrowService sets CopyStatus.Available on return	✓ Done
18	View pending fines	FineService.GetFineSummaryAsync() using PostgreSQL function	✓ Done
19	Pay fine	FineService.PayFineAsync() with amount validation	✓ Done
20	View fine history	FineService.GetFineHistoryAsync()	✓ Done
21	PostgreSQL function	calculate_member_fine() called from FineRepository	✓ Done

#	Requirement	Implementation	Status
22	EF Core transaction	BeginTransactionAsync() in BorrowBookAsync/ReturnBookAsync	✓ Done
23	Reports	7 reports implemented in ReportMenu	✓ Done
24	Exception handling	10 custom exception classes with try-catch in all menus	✓ Done
25	Input validation	InputValidator with regex for phone, email, name etc.	✓ Done

7.2 Test Coverage

Test Class	Tests	Coverage Area
InputValidatorTests	30+	Regex validation for phone, email, name, ID, amount
MemberServiceTests	15+	Add, search, deactivate, status update, duplicates
BookServiceTests	20+	Add book/copy, search, category, copy status, available count
BorrowServiceTests	9+	All 6 borrow validation rules, return flow, fine calculation
FineServiceTests	12+	Fine summary, pay fine, history, borrow-specific payment
Total	86+	Full business layer coverage

8. NuGet Packages

Package	Version	Project	Purpose
Microsoft.EntityFrameworkCore	8.0.0	DataAccessLayer	Core ORM framework
Npgsql.EntityFrameworkCore.PostgreSQL	8.0.0	DataAccessLayer	PostgreSQL EF Core provider
Microsoft.EntityFrameworkCore.Design	8.0.0	DataAccessLayer	Scaffolding and migrations
Microsoft.Extensions.DependencyInjection	8.0.0	PresentationLayer	DI container
Microsoft.Extensions.Configuration	8.0.0	PresentationLayer	Configuration management
Microsoft.Extensions.Configuration.Json	8.0.0	PresentationLayer	JSON config support
Microsoft.EntityFrameworkCore.Design	8.0.0	PresentationLayer	EF tools startup project
xunit	Latest	Tests	Testing framework
Moq	4.20.70	Tests	Mock object library
FluentAssertions	6.12.0	Tests	Fluent assertion syntax
Microsoft.EntityFrameworkCore.InMemory	8.0.0	Tests	In-memory DB for tests

9. Console Menu Structure

Menu Option	Sub-Options
1. Member Management	Add Member View All Search Email Search Phone Update Status Deactivate View Lendings
2. Book Management	Add Book Add Copy View All Search By Category Update Copy Status Add Category View Categories
3. Borrow Book	Enter Member ID → Enter Book ID → Validate all rules → Confirm borrow
4. Return Book	Enter Member ID → View active borrows → Select Borrow ID → Pay fine if overdue → Confirm return
5. Fine Management	View Pending Fine Pay Fine View Fine History
6. Reports	Borrowed Books Overdue Member Summary By Category Pending Fines Most Borrowed History
7. Exit	Display log file path and exit application